מערכות הפעלה - 20594

סמסטר 2020ב

מרכז ההוראה: דוד שריאל

התחיל ב:	8/07/2020, 13:00
מצב	הסתיים
הושלם ב-	8/07/2020, 16:04
הזמן שלקח	3 שעות 4 דקות

מידע

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. **המבחן מורכב משלושה חלקים.**
2. **בחלקים א' ו ב' מופיעות שאלות פתוחות. את התשובות לכל השאלות הפתוחות יש לכתוב בתיבת טקסט .ענו תשובות מלאות בקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות. אין לחרוג מהמקום המוקצה.**
3. **בחלק ג' (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות.**
4. **משך הבחינה - שלוש שעות.**

בהצלחה!

שאלה **1**

הושלם

ניקוד השאלה: 20.00

במערכת מקבילית ישנם T תהליכונים (threads), כאשר T הוא מספר קבוע קטן מ 1024. במערכת מספר מעבדים. בכל עת התהליכונים יכולים להיות מועברים למעבד אחר (לא חשוב כעת לפי איזה קריטריון). הבעיה היא שהתהליכונים יכולים "ליפול" בזמן לא ידוע ומסיבה לא ידועה. כאשר תהליכון "נופל" הוא מפסיק לבצע את ההוראות שלו ולא חוזר לעולם.

תהליכונים יכולים לבצע פעולות שדורשות מתהליכונים אחרים להמתין. הפעולות הללו מוגבלות בזמן וחסומות ע"י הזמן שלוקח לבצע את הקוד הבא:

```
int counter;
for (i=0; i++; i<MAX_INT) {
        counter++;
}
```

נרצה לנצל את העובדה הזאת כדי לזהות נפילת תהליכונים במערכת המקבילית, הרי אם תהליכון כלשהו מחזיק במנעול לאורך הזמן שעולה על זמן הביצוע של הלולאה דלעיל – הרי שזה אומר שהתהליכון "נפל".

הוצעו 2 הצעות לפתור את הבעיה בעזרת מנעול אשר נפתח בעצמו אם הוא תפוס יותר מדי זמן.

ההצעה הראשונה הייתה כדלהלן:

```
typedef struct Lock_t
{
        bool    b;
} Lock;
```

```
void down(Lock* lock) {
    int counter = 0;
    while ( !t&s( &( lock →b) ) ) {
        counter++;
        if (counter == MAX_INT) {
            up(lock);
        }
    } // end of while
}
```

```
void    up (Lock* lock) {
    bzero(lock→b);
}
```

ההצעה השניה הייתה:

```
typedef struct Lock_t
{
        bool      b;
        int       counter;
} Lock;
```

```
void down(Lock* lock) {
    int oldCounter;
    while ( !t&s( &(lock→b) ) ) {
        oldCounter =
            fetch&Increment( &(lock→counter) );
        if ( (oldCounter +1)  == MAX_INT) {
                up(lock);
        }
    } // end of while
    lock→ counter = 0;
}
```

```
void up(Lock* lock)
{
    lock→b = 0;
    lock→counter = 0;
}
```

הבהרות:

- **הניחו ש-t&s היא פקודה אטומית אשר קוראת ערך מכתובת ואם הוא אפס משנה אותו באופן אטומי ל-1. הפקודה מחזירה true אם הצליחה לשנות ערך. אחרת היא מחזירה false.**

- **הניחו ש- fetch&Increment היא פקודה אטומית אשר קוראת ערך מכתובת, מקדמת את הערך ב-1 ואחרי כן כותבת אותו בחזרה וכל זה באופן אטומי. הפקודה מחזירה את הערך הישן שנקרא (לפני עדכון).**

סעיף א (10 נקודות)
הסבירו מדוע ההצעה הראשונה יכולה לגרום לכך שלא תתקיים מניעה הדדית (mutual exclusion).

סעיף ב (10 נקודות)
הוכיחו או הפריכו טענה הבאה: "ההצעה השניה גם כן סובלת מבעיה של חוסר מניעה הדדית".

א. בגלל ה-count++. אם יהיה CSW, אזי יקודם פעמיים ובבדיקה אחרי זה לא ניכנס פנימה.

ב. הטענה נכונה: גם כאן אפשרי, כאשר יש CSW אחרי השמה של oldCounter, כלומר רגע לפני ה-if.

שאלה **2**

הושלם

ניקוד השאלה: 23.00

א. (4 נקודות)

מהו מספר שורות זיכרון המטמון בהן נדרש לערוך חיפוש מקבילי במקרה של זיכרון אסוציאטיבי באופן מלא (fully associative)? נמקו.

ב. (4 נקודות)
מהו מספר שורות זיכרון המטמון בהן נדרש לערוך חיפוש מקבילי במקרה של זיכרון אסוציאטיבי בעל מיפוי ישיר (direct mapping)? נמקו.

ג. (5 נקודות)
תארו את אלגוריתם ה-best fit לניהול שטחי זיכרון באמצעות רשימה משורשרת.

ד. (5 נקודות)
איזה סוג של ריסוק (fragmentation) מייצר אלגוריתם ה-best fit לניהול שטחי זיכרון באמצעות רשימה משורשרת ומהי השפעתו על מפת השטחים הפנויים/התפוסים?

ה. (5 נקודות)
דרגו את האלגוריתמים לפינוי דפים לפי ביצועים במובן של יכולת חיזוי בסדר יורד: אופטימלי, FIFO, אלגוריתם הזדמנות שנייה, Least Recently Used, WSClock.

א. בכל השורות יחד במקביל - זו המשמעות של FULL.

ב. שורה אחת - זו המשמעות של DIRECT. במיפוי ישיר כל שורה ממפה שאופן חד ערכי.

ג. חייב לעבור על כל הרשימה כדי לחפש את החור הקטן ביותר שמספיק גדול כדי להכניס לשם את התהליך. לא יעיל בכלל וגם בעייתי בגלל יצירת ריסוקים קטנטנים.

ד. ריסוק חיצוני. מלא שטחים פנויים קטנטנים שאי אפשר לדחוף לשם כלום, לכן יהיה צורך לצופף את השטח בעיתים קרובות.

ה. דירוג מהכי טוב למעלה ועד הכי פחות טוב למטה:
אופטימלי
WSClock
Least Recently Used
אלגוריתם הזדמנות שנייה
FIFO

שאלה **3**

הושלם

ניקוד השאלה: 12.00

יהי

- **s –גודל זיכרון של תהליך (גודל ממוצע במערכת נתונה)**
- **p– גודל הדף**
- **e – גודל של שורה בטבלת הדפים של תהליך**

ציינו מה יהיה

- **המספר המשוער של דפים שהתהליך זקוק להם.**
- **הגודל של טבלת דפים של תהליך.**
- **הבזבוז עקב הריסוק הפנימי בדף האחרון של התהליך.**

חשבו את התקורה הכוללת של השימוש בזיכרון וירטואלי והסיקו ממנה מהו הגודל האופטימלי של הדף.

approximate number of pages = s/p
page table size = s*e/p
wasted memory = p/2
overhead = s*e/p + p/2
p = sqrt(2*s*e)

שאלה **4**

הושלם

ניקוד השאלה: 5.00

בפונקציה thread_create האחריאת על יצירת תהליכונים במערת xv6 שומרים 4 בתים על מחסנית תהליכון עבור כתובת חזרה. למה נועדה הכתובת הזאת ומי אחרי לשמור אותה במקום המיועד לכך במחסנית?

```
63  void
64  thread_create(void (*func)())
65  {
66    thread_p t;
67
68    for (t = all_thread; t < all_thread + MAX_THREAD; t++) {
69      if (t->state == FREE) break;
70    }
71    t->sp = (int) (t->stack + STACK_SIZE);   // set sp to the top of the stack
72    t->sp -= 4;                              // space for return address
```

פונקציה זו צריכה (אחראית) לשמור את הכתובת, איפשהו אחרי שהיא סידרה את המקום לכך.
אם אני לא טועה, כאן זה צריך להיות המצביע לפונקציית ריצת התהליכון, אשר קיבלנו כפרמטר. וככה יודעים איזו פונקציה להפעיל כשיוצרים את התהליכון.
באופן כללי, כאשר עושים RET בסיום פונקציה, שולפים את ה-return address מהמחסנית וככה יודעים להכניס אותו ל-PC (הפקודה הבאה לביצוע) כדי לחזור לריצה מאיפה שבאנו.

שאלה **5**

הושלם

ניקוד השאלה: 5.00

לפניכם קטע קוד מתוך פונקציית runCommand של תוכנת shell.c ממ"ן 12.

```
371         signal (SIGINT, SIG_DFL);
372         signal (SIGQUIT, SIG_DFL);
373         signal (SIGTSTP, SIG_DFL);
374         signal (SIGTTIN, SIG_DFL);
375         signal (SIGTTOU, SIG_DFL);
376         signal (SIGCHLD, SIG_DFL);
377
378         setpgid(newJob.progs[i].pid, newJob.progs[0].pid);
379
380         execvp(newJob.progs[i].argv[0], newJob.progs[i].argv);
381             fprintf(stderr, "exec() of %s failed: %s\n",
382                     newJob.progs[i].argv[0],
383                     strerror(errno));
384             exit(1);
385         }
```

הסבירו מדוע בשורות 371-376 מחזירים את ה signal disposition של הסיגנלים הנ"ל ל SIG_DEF.

זה קטע קוד עבור ה-CHILD כדי שיוכל לקבל סיגנלים.
עבור ה-PARENT חסמנו סיגנלים אלו כדי שלא יתפוס אותם הוא עצמו ויעצור.
כיוון שבהולדת הבן, הבן קיבל אותם בירושה מהאבא כחסומים, בבן חייבים להחזיר אותם למקור (ברירית המחדל).

שאלה **6**

הושלם

ניקוד השאלה: 5.00

האם קריאה לפונקציה itrunc מתוך הקובץ fs.c במערכת הפעלה xv6 יכולה לגרום לפניה פיזית לדיסק בשורה 429? הסבירו את תשובתכם.

```
409  // Truncate inode (discard contents).
410  // Only called when the inode has no links
411  // to it (no directory entries referring to it)
412  // and has no in-memory reference to it (is
413  // not an open file or current directory).
414  static void
415  itrunc(struct inode *ip)
416  {
417    int i, j;
418    struct buf *bp;
419    uint *a;
420
421    for(i = 0; i < NDIRECT; i++){
422      if(ip->addrs[i]){
423        bfree(ip->dev, ip->addrs[i]);
424        ip->addrs[i] = 0;
425      }
426    }
427
428    if(ip->addrs[NDIRECT]){
429      bp = bread(ip->dev, ip->addrs[NDIRECT]);
430      a = (uint*)bp->data;
431      for(j = 0; j < NINDIRECT; j++){
432        if(a[j])
433          bfree(ip->dev, a[j]);
434      }
435      brelse(bp);
436      bfree(ip->dev, ip->addrs[NDIRECT]);
437      ip->addrs[NDIRECT] = 0;
438    }
```

כן.
אנחנו מסתכלים על המצביע לבלוק של single indirect.
לכן, ייתכן ונצטרך לטעון אותו מהדיסק.

שאלה **7**

הושלם

ניקוד השאלה: 5.00

האם ניתן לממש בצורה יעילה פעולות עבודה עם קבצים

open, close, read, write, append, seek

וזאת ללא טבלת open file descriptors המוחזקת ע"י תהליך במערכת הפעלה לינוקס? נמקו את תשובתכם.

המנגנון של FD מאפשר להשקיע מאמץ גדול ב-OPEN, אבל משם להמשיך לעבוד עם FD ישירות ובמהירות עבור הקריאות האחרות.
כמו כן, ביצירת בנים, הבנים מקבלים בירושה את הטבלה הזו והחיים שלהם דבש.
ללא הטבלה של הקבצים הפתוחים, יהיה צורך בסריקות מלאות.

שאלה **8**

הושלם

ניקוד השאלה: 5.00

מה הן 3 קבוצות תהליכונים (threads) אשר מערכת הפעלה לינוקס מבדילה בינהן לצורך תזמון תהליכונים?

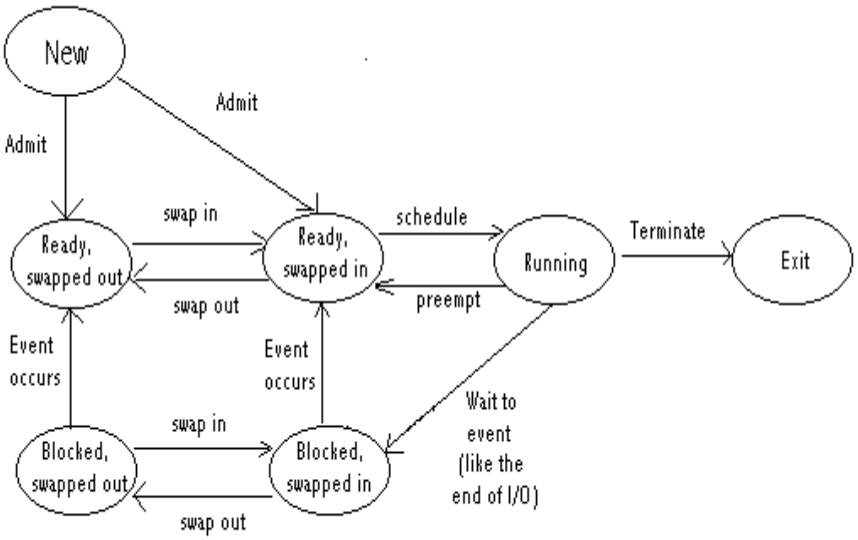
Real-time FIFO
Real-time round robin
Timesharing

שאלה **9**

הושלם

ניקוד השאלה: 5.00

לגרף עם שלושת המצבים הבסיסיים של התהליך במערכת ההפעלה הוספנו עוד כמה מצבים:



עדיפותו של תהליך P שנמצא במצב "Ready, swapped out" גבוהה מעדיפותם של כל אחד מהתהליכים שנמצאים במצב "Ready, swapped in". האם ייתכן מצב שמנגנון התזמון ישאיר את התהליך P במצב "Ready, swapped out" וימשיך לתזמן תהליכים שנמצאים בזיכרון מבלי לבצע swap in ל-P?

יש לבחור תשובה אחת:

- **כן. המצב ייתכן, למשל משיקולי גודל התהליך P אשר צריך להביאו לזיכרון.**
- **כן. המצב ייתכן כאשר P שוהה זמן רב במצב "Ready, swapped out".**
- **לא. אין שום סיבה המצדיקה מהלך כזה.**
- **כן. המצב ייתכן כאשר יש חשיבות מכרעת לתזמון תהליכים עם עדיפות גבוהה על פני שיקולים אחרים.**

שאלה **10**

הושלם

ניקוד השאלה: 5.00

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה -I node.

- **גודל הבלוק במערכת הקבצים הוא 2 Kbytes**
- **כתובת הבלוק היא 8 בתים (bytes)**
- **10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק**
- **שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block**
- **עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block**
- **ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block**

חשבו מהו הגודל המקסימלי של קובץ במערכת הקבצים הזו?

יש לבחור תשובה אחת:

- Gbytes 30
- Gbytes 31
- Gbytes 32
- Gbytes 33

שאלה **11**

הושלם

ניקוד השאלה: 5.00

התנאי לכך שיהיה ניתן לתזמן n משימות מחזוריות במערכת זמן אמת (real-time) הוא:

$$\sum_{i=1}^{n} C_i / P_i \leq 1$$

כאשר Pi הוא זמן המחזור (period) ו-Ci הוא זמן ה-CPU של המשימה ה-i-ית. איזו הנחה סמויה מסתתרת בנוסחה?

יש לבחור תשובה אחת:

- **זמן החלפת המשימות (context switch) הוא זניח.**
- **לכל i בין 1 ל-n מתקיים Pi= Ci.**
- **לא ניתן לתזמן אף משימה כל עוד לא מתקיים שוויון בנוסחה הנ"ל.**
- **לכל i בין 1 ל-n מתקיים כי Pi < Ci.**

שאלה **12**

הושלם

ניקוד השאלה: 5.00

באיזו שכבה של תוכנת קלט/פלט ממומש לרוב מנגנון הגנה על קבצים?

יש לבחור תשובה אחת:

- **בתוך תוכניות התיאום בין ההתקן לבין מערכת ההפעלה (device drivers).**
- **בתוך תוכנת קלט/פלט הבלתי תלויה בהתקן (device independent software).**
- **במערכת הטיפול בפסיקות הנוצרות על-ידי ההתקנים (interrupt handling mechanism).**
- **בשדים (daemons) לארגון הפלט ובפונקציות הספרייה שאינן תלויות חומרה.**